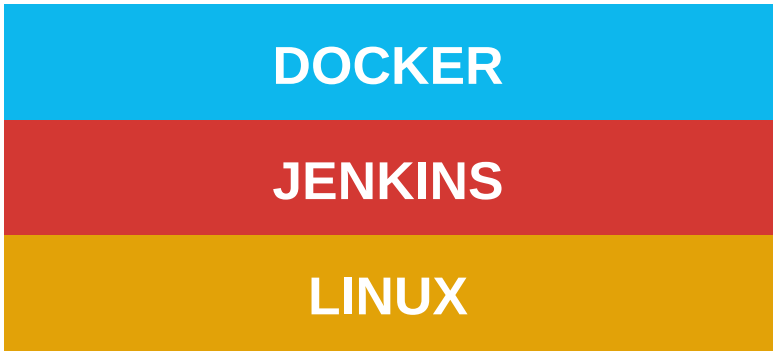
THE ULTIMATE

# DevOps Cheat Sheet

Docker • Jenkins • Linux




Quick-reference commands and syntax for everyday use — no fluff, just what you need.

# LINUX CHEAT SHEET

## 1. File & Directory Navigation

| Command | What it does |
|---|---|
| `pwd` | Print current working directory. |
| `ls -la` | List all files (incl. hidden) with details. |
| `cd /path/to/dir` | Change directory. |
| `cd ..` | Go up one directory level. |
| `cd ~` | Go to home directory. |
| `mkdir myfolder` | Create a new directory. |
| `mkdir -p a/b/c` | Create nested directories at once. |
| `rmdir myfolder` | Remove an empty directory. |
| `rm -rf myfolder` | Force-remove a directory and its contents (careful!). |
| `tree` | Show directory structure as a tree. |

## 2. File Operations

| Command | What it does |
|---|---|
| `touch file.txt` | Create an empty file / update timestamp. |
| `cp a.txt b.txt` | Copy a file. |
| `cp -r dir1 dir2` | Copy a directory recursively. |
| `mv a.txt b.txt` | Move or rename a file. |
| `rm file.txt` | Delete a file. |
| `cat file.txt` | Print file contents. |
| `less file.txt` | View file page-by-page (q to quit). |
| `head -n 10 file.txt` | Show first 10 lines. |
| `tail -n 10 file.txt` | Show last 10 lines. |
| `tail -f app.log` | Follow a log file live (real-time updates). |
| `find / -name '*.log'` | Search for files by name across the system. |
| `locate filename` | Fast file search using an index (needs updatedb). |

## 3. Permissions & Ownership

| Command | What it does |
|---|---|
| `chmod 755 file.sh` | Set permissions: owner rwx, group/others rx. |
| `chmod +x script.sh` | Make a file executable. |
| `chown user:group file` | Change file owner and group. |
| `chown -R user:group dir/` | Recursively change ownership. |
| `umask` | Show default permission mask for new files. |

**Permission digits:** 4=read, 2=write, 1=execute (add them up). e.g. 7 = rwx, 6 = rw-, 5 = r-x.

## 4. Process Management

| Command | What it does |
|---|---|
| `ps aux` | List all running processes. |

| | |
|---|---|
| `top` | Live view of processes/CPU/memory usage. |
| `htop` | Improved, colorful version of top (may need install). |
| `kill PID` | Terminate a process by ID (graceful). |
| `kill -9 PID` | Force-kill a process. |
| `killall processname` | Kill all processes matching a name. |
| `jobs` | List background jobs in current shell. |
| `bg / fg` | Resume a job in background / foreground. |
| `nohup cmd &` | Run a command immune to hangups, in background. |

## 5. Networking

| Command | What it does |
|---|---|
| `ifconfig / ip a` | Show network interfaces and IP addresses. |
| `ping google.com` | Test connectivity to a host. |
| `curl https://url` | Fetch a URL's content from command line. |
| `wget https://url/file` | Download a file from the web. |
| `netstat -tulnp` | Show listening ports and related processes. |
| `ss -tulnp` | Modern replacement for netstat. |
| `scp file user@host:/path` | Copy a file to a remote server over SSH. |
| `ssh user@host` | Connect to a remote machine via SSH. |
| `ssh -i key.pem user@host` | SSH using a specific private key. |

## 6. Package Management

| Command | What it does |
|---|---|
| `apt update && apt upgrade` | Update package list & upgrade installed packages (Debian/Ubuntu). |
| `apt install pkgname` | Install a package (Debian/Ubuntu). |
| `apt remove pkgname` | Remove a package. |
| `yum install pkgname` | Install a package (CentOS/RHEL). |
| `dnf install pkgname` | Install a package (Fedora/newer RHEL). |
| `snap install pkgname` | Install a snap package (universal Linux package). |

# LINUX CHEAT SHEET (contd.)

## 7. Text Processing (grep, sed, awk)

| Command | What it does |
|---|---|
| `grep 'error' file.log` | Search for lines containing 'error'. |
| `grep -i 'error' file.log` | Case-insensitive search. |
| `grep -r 'TODO' .` | Recursively search all files in current dir. |
| `grep -v 'debug' file.log` | Show lines NOT containing 'debug'. |
| `sed 's/old/new/g' file` | Replace all 'old' with 'new' in file (prints to screen). |
| `sed -i 's/old/new/g' file` | Replace in-place (edits the actual file). |
| `awk '{print $1}' file` | Print the first column of each line. |
| `sort file.txt` | Sort lines alphabetically. |
| `sort -n file.txt` | Sort lines numerically. |
| `uniq file.txt` | Remove adjacent duplicate lines. |
| `wc -l file.txt` | Count number of lines in a file. |
| `cut -d',' -f1 file.csv` | Extract 1st field from a comma-separated file. |

## 8. Disk & System Info

| Command | What it does |
|---|---|
| `df -h` | Show disk space usage (human-readable). |
| `du -sh folder/` | Show total size of a folder. |
| `free -h` | Show RAM/swap usage. |
| `uname -a` | Show kernel & system info. |
| `uptime` | Show how long the system has been running. |
| `lscpu` | Show CPU details. |
| `history` | Show command history. |
| `whoami` | Show current logged-in user. |
| `date` | Show current date and time. |

## 9. Users & Groups

| Command | What it does |
|---|---|
| `adduser john` | Create a new user (interactive). |
| `passwd john` | Set/change a user's password. |
| `usermod -aG sudo john` | Add user 'john' to the sudo group. |
| `deluser john` | Delete a user. |
| `su - john` | Switch to another user's session. |
| `sudo command` | Run a single command with admin (root) privileges. |

## 10. Archives & Compression

| Command | What it does |
|---|---|
| `tar -cvf out.tar dir/` | Create a tar archive from a directory. |
| `tar -xvf out.tar` | Extract a tar archive. |

| | |
|---|---|
| `tar -czvf out.tar.gz dir/` | Create a gzip-compressed tar archive. |
| `tar -xzvf out.tar.gz` | Extract a .tar.gz archive. |
| `zip -r out.zip dir/` | Create a zip archive. |
| `unzip out.zip` | Extract a zip archive. |

## 11. I/O Redirection & Pipes

| Command | What it does |
|---|---|
| `cmd > file.txt` | Redirect output to file (overwrite). |
| `cmd >> file.txt` | Redirect output to file (append). |
| `cmd 2> err.txt` | Redirect only error output to file. |
| `cmd1 \| cmd2` | Pipe output of cmd1 as input to cmd2. |
| `cmd < input.txt` | Use file contents as input to a command. |

# DOCKER CHEAT SHEET

## 1. Images

| Command | What it does |
|---|---|
| `docker pull nginx` | Download an image from Docker Hub. |
| `docker images` | List all local images. |
| `docker build -t myapp:1.0 .` | Build an image from a Dockerfile in current dir. |
| `docker rmi myapp:1.0` | Remove an image. |
| `docker tag myapp:1.0 repo/myapp:1.0` | Tag an image for pushing to a registry. |
| `docker push repo/myapp:1.0` | Push an image to a registry (e.g. Docker Hub). |
| `docker history myapp:1.0` | Show the layers/history of an image. |
| `docker save -o myapp.tar myapp:1.0` | Export an image to a .tar file. |
| `docker load -i myapp.tar` | Import an image from a .tar file. |

## 2. Containers

| Command | What it does |
|---|---|
| `docker run nginx` | Run a container from an image. |
| `docker run -d -p 8080:80 nginx` | Run detached, mapping host port 8080 to container port 80. |
| `docker run -it ubuntu bash` | Run interactively with a terminal (bash shell). |
| `docker run --name web -d nginx` | Run a container with a custom name. |
| `docker ps` | List running containers. |
| `docker ps -a` | List all containers (including stopped). |
| `docker stop container_id` | Stop a running container. |
| `docker start container_id` | Start a stopped container. |
| `docker restart container_id` | Restart a container. |
| `docker rm container_id` | Remove a stopped container. |
| `docker rm -f container_id` | Force-remove a running container. |
| `docker logs container_id` | View a container's logs. |
| `docker logs -f container_id` | Follow container logs live. |
| `docker exec -it container_id bash` | Open a shell inside a running container. |
| `docker inspect container_id` | Show detailed JSON info about a container. |
| `docker stats` | Live stream of container resource usage. |
| `docker cp file.txt container_id:/app/` | Copy a file into a running container. |

## 3. Dockerfile Instructions (Quick Reference)

| Command | What it does |
|---|---|
| `FROM ubuntu:22.04` | Base image to build from. |
| `WORKDIR /app` | Set the working directory inside the container. |
| `COPY . .` | Copy files from host into the image. |

| | |
|---|---|
| `ADD archive.tar.gz /app` | Like COPY, but can also auto-extract archives / fetch URLs. |
| `RUN apt-get install -y curl` | Run a command at build time (creates a new layer). |
| `ENV NODE_ENV=production` | Set an environment variable. |
| `EXPOSE 3000` | Document which port the container listens on. |
| `CMD ["node", "app.js"]` | Default command run when container starts (overridable). |
| `ENTRYPOINT ["python3"]` | Fixed command that always runs (args appended to it). |
| `VOLUME /data` | Declare a mount point for persistent data. |
| `ARG VERSION=1.0` | Build-time variable (not available at runtime). |

# DOCKER CHEAT SHEET (contd.)

## 4. Volumes (Persistent Data)

| Command | What it does |
|---|---|
| `docker volume create mydata` | Create a named volume. |
| `docker volume ls` | List all volumes. |
| `docker volume inspect mydata` | Show details about a volume. |
| `docker volume rm mydata` | Remove a volume. |
| `docker run -v mydata:/app/data nginx` | Mount a named volume into a container. |
| `docker run -v $(pwd):/app nginx` | Mount current host directory (bind mount). |

## 5. Networking

| Command | What it does |
|---|---|
| `docker network ls` | List all Docker networks. |
| `docker network create mynet` | Create a custom network. |
| `docker network inspect mynet` | Show details about a network. |
| `docker run --network=mynet nginx` | Attach a container to a specific network. |
| `docker network connect mynet container_id` | Connect a running container to a network. |

## 6. Docker Compose

| Command | What it does |
|---|---|
| `docker compose up` | Start all services defined in docker-compose.yml. |
| `docker compose up -d` | Start services in detached (background) mode. |
| `docker compose down` | Stop and remove all services, networks. |
| `docker compose ps` | List running services in the compose project. |
| `docker compose logs -f` | Follow logs of all services. |
| `docker compose build` | Build/rebuild images defined in the compose file. |
| `docker compose exec web bash` | Open a shell inside the 'web' service container. |

## Example docker-compose.yml

```
version: '3.8'
services:
  web:
    build: .
    ports:
      - "8080:80"
    volumes:
      - ./app:/usr/share/nginx/html
    depends_on:
      - db
  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: secret
```

```
    volumes:
      - db_data:/var/lib/mysql
volumes:
  db_data:
```

## 7. Cleanup Commands

| Command | What it does |
| --- | --- |
| `docker system df` | Show disk space used by Docker (images/containers/volumes). |
| `docker container prune` | Remove all stopped containers. |
| `docker image prune` | Remove unused (dangling) images. |
| `docker image prune -a` | Remove ALL unused images (not just dangling). |
| `docker volume prune` | Remove all unused volumes. |
| `docker system prune -a --volumes` | Nuke everything unused: containers, images, networks, volumes. |

# JENKINS CHEAT SHEET

## 1. Declarative Pipeline Skeleton

```
pipeline {
    agent any
    environment {
        APP_ENV = 'production'
    }
    options {
        timeout(time: 30, unit: 'MINUTES')
    }
    stages {
        stage('Checkout') {
            steps { git 'https://github.com/example/app.git' }
        }
        stage('Build') {
            steps { sh 'mvn clean package' }
        }
        stage('Test') {
            steps { sh 'mvn test' }
        }
        stage('Deploy') {
            when { branch 'main' }
            steps { sh './deploy.sh' }
        }
    }
    post {
        success { echo 'Build succeeded!' }
        failure { echo 'Build failed.' }
        always  { cleanWs() }
    }
}
```

## 2. Core Declarative Directives

| Command | What it does |
|---|---|
| `agent any / agent none / agent { label 'x' }` | Where the pipeline (or stage) runs. |
| `stages { stage('Name') { ... } }` | Container for one or more named build stages. |
| `steps { sh '...' }` | The actual commands executed inside a stage. |
| `environment { VAR = 'value' }` | Define environment variables for the pipeline/stage. |
| `parameters { string(name:'V', ...) }` | Ask for user input before the build starts. |
| `triggers { cron('H 2 * * *') }` | Schedule automatic builds (cron-style). |
| `when { branch 'main' }` | Conditionally run a stage. |
| `parallel { stage(...) stage(...) }` | Run multiple stages simultaneously. |
| `post { success{} failure{} always{} }` | Actions to run after pipeline/stage completes. |
| `options { timeout(...) retry(2) }` | Pipeline-wide behavior settings. |
| `tools { maven 'M3' jdk 'JDK17' }` | Auto-install/use pre-configured build tools. |

## 3. Common Pipeline Steps

| Command | What it does |
|---|---|
| `sh 'command'` | Run a shell command (Linux/Mac agents). |
| `bat 'command'` | Run a Windows batch command. |

| | |
|---|---|
| `echo 'message'` | Print a message to the console log. |
| `git url: '...', branch: 'main'` | Checkout code from a Git repository. |
| `checkout scm` | Checkout using the SCM configured for the job. |
| `junit 'reports/*.xml'` | Publish JUnit test result reports. |
| `archiveArtifacts 'build/*.jar'` | Save build output files for later download. |
| `input message: 'Deploy?'` | Pause and wait for manual approval. |
| `withCredentials([...]) { }` | Safely inject secrets into a block of steps. |
| `retry(3) { sh '...' }` | Retry a step up to N times if it fails. |
| `timeout(time: 5, unit: 'MINUTES') { }` | Fail a block if it exceeds the given time. |
| `cleanWs()` | Clean the workspace directory. |

# JENKINS CHEAT SHEET (contd.)

## 4. Docker Agent Syntax

```
pipeline {
    agent {
        docker { image 'node:20-alpine' }
    }
    stages {
        stage('Test') {
            steps { sh 'npm install && npm test' }
        }
    }
}
```

## 5. Shared Library Usage

| Command | What it does |
|---|---|
| `@Library('my-lib') _` | Import a global shared library at the top of a Jenkinsfile. |
| `vars/myStep.groovy` | File defining a reusable global function (e.g. myStep()). |
| `myStep('arg1')` | Call a shared library function inside any pipeline. |

## 6. Useful Built-in Environment Variables

| Command | What it does |
|---|---|
| `env.BUILD_NUMBER` | Current build's number. |
| `env.JOB_NAME` | Name of the current job/pipeline. |
| `env.WORKSPACE` | Path where code was checked out. |
| `env.BRANCH_NAME` | Current Git branch (multibranch pipelines). |
| `env.BUILD_URL` | Direct link to this build's page. |
| `env.GIT_COMMIT` | SHA of the checked-out commit. |

## 7. CLI & Server Management

| Command | What it does |
|---|---|
| `java -jar jenkins.war` | Run Jenkins directly (standalone WAR file). |
| `systemctl start jenkins` | Start Jenkins service (Linux). |
| `systemctl status jenkins` | Check Jenkins service status. |
| `systemctl restart jenkins` | Restart the Jenkins service. |
| `cat /var/lib/jenkins/secrets/initialAdminPassword` | Get the first-time setup unlock key. |
| `java -jar jenkins-cli.jar -s http://host:8080/ build JOB` | Trigger a build via CLI. |
| `java -jar jenkins-cli.jar -s http://host:8080/ list-jobs` | List all jobs via CLI. |

## 8. Triggers Quick Reference

| Command | What it does |
|---|---|
| `pollSCM('H/5 * * * *')` | Check Git for changes every 5 minutes. |
| `cron('H 2 * * *')` | Run the build daily around 2 AM. |

| | |
|---|---|
| `upstream(upstreamProjects: 'jobA', threshold: hudson.model.Result.SUCCESS)` | Trigger after another job succeeds. |
| `GitHub webhook` | Instantly trigger build on git push (needs 'GitHub hook trigger' enabled). |

---

*Keep this handy while working — bookmark the commands you use most often!*